

C++プログラミングI

- 第3回：string とvector
- 担当：二瓶芙巳雄

複数の値を扱うデータ型

- 配列：複数の値をまとめて格納するデータ構造
- C言語の配列：
 - C++でも使用できる
 - 最低限の管理機能しか提供されていない
 - 使用するには様々な知識必要
 - ポインタとメモリ管理
- C++の配列：
 - **string**型: 文字列を扱うためのデータ型
 - **vector**型: C配列を拡張した汎用の配列

string型

string 型

- `string` 型：文字列を扱うための型
基本データ型ではなく，標準ライブラリに定義されたデータ型
- `<string>` ヘッダファイルをインクルード
- ※ C言語で文字列を扱うには，`char` 型の配列 (C-string) を使っていた
- 文字列リテラルはC-string
 - ヌル文字で終端する配列 (p.8)
 - 文字が並ぶ範囲をどう表すかが重要

```
1  #include <iostream>
2  #include <string> // iostreamがあるなら省略可
3
4  int main() {
5      // 文字列リテラルで string型変数を初期化
6      std::string s {"Hello!"};
7      std::cout << s << "\n";    // cout の利用
8  }
```

```
% ./a.out
Hello!
```

string: 初期化, 代入, 入力

- `string` の初期化や代入に文字列リテラルを使用する
- 文字列リテラルは `string` 型ではない
 - 初期化・代入時に, 型変換されている

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string s {"Hello!"}; // 文字列リテラル
6      std::string s2 {s};       // 他変数の初期値
7      std::string s3;          // "" (空文字列) での初期化
8
9      s = "How are you?";      // 文字列リテラルの代入
10     s3 = s2;                 // 代入, c言語配列では不可
11
12     std::cin >> s2;          // cin も利用できる
13 }
```

string: 連結

- `+` 演算子は文字列の連結
- `+=` 演算子で更新も可能
- 以下に注意 (`s` は `string` 変数とする)
 - 不可: `"abc" + "xyz"` (リテラルのみ連結)
 - 不可: `s += "abc" + "xyz"`
 - 可能: `s + "abc" + "xyz"`
 - 可能: `"abc" + s + "xyz"`
 - これはどうなるか?: `"abc" + "xyz" + s`

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string s2 {"Hello"};
6      std::string s3 {};
7
8      s3 = s2 + "! ";           // 文字列リテラル
9      s3 = s3 + s3;             // 変数
10     std::cout << s3 << "\n";
11
12     s2 += ", thank you, ";     // 文字列の追加
13     s3 = " and You";
14     s2 += s3;                  // 変数の追加
15     s2 += '?';                 // 文字の追加
16     std::cout << s2 << "\n";
17 }
```

```
% ./a.out
Hello! Hello!
Hello, thank you, and You?
```

string 型変数の比較

- 等値比較の他に順序比較もできる
- 順序比較は先頭の文字からASCII表で比較
 - `abcd < abxa` → `true` . `c < x` であるため
 - `make < maker` → `true` . `make` まで同じだが, `4文字 < 5文字` であるため

```
1  #include <iostream>
2
3  int main() {
4      std::string x, ans{"C++"};
5      std::cin >> x;
6
7      if (x == ans) std::cout << " correct!\n";
8      else if (x == "CPU") std::cout << " no...\n";
9      else std::cout << " incorrect...\n";
10
11     std::string a {"abc"};
12     if (a < "xyz") std::cout << a << " < xyz is true\n";
13 }
```

```
% ./a.out
C++
correct!
abc < xyz is true
% ./a.out
CPU
no...
abc < xyz is true
% ./a.out
hello
incorrect...
abc < xyz is true
```

※下線部はキーボード入力

半開区間

- C++で良く使われる数学用語
- $[a, b)$ を離散値で用いる（数学では実数が対象）

名前	表記法	要素の条件
閉区間	$[a, b]$	$a \leq x \leq b$
开区間	(a, b)	$a < x < b$
半开区間（左開右閉）	$(a, b]$	$a < x \leq b$
半开区間（左閉右開）	$[a, b)$	$a \leq x < b$

string 型変数内の文字へのアクセス

- 添字の利用（空文字列でない場合）
 - 配列の添字を使って変数内の文字にアクセス
 - n文字の文字列の添字は半開区間 `[0, n)`

メンバ関数 動作

`s.empty()` `s` の長さが0であれば `true` を返す

`s.size()` `s` の長さを返す

`s.front()` `s` の先頭の文字を返す

`s.back()` `s` の末尾の文字を返す

- メンバ関数（対象変数用の処理を行う関数）
 - `s` はstring型の変数とする

```
1  #include <iostream>
2  int main() {
3      std::string s {"abcXefg"};
4      char ch { s[3] }; // 右辺値として. [i]でi番目の文字にアクセス
5      s[3] = 'd';       // 左辺値として
6      std::cout << ch << " " << s << " " << "\n";
7
8      std::string t;
9      std::cin >> t;
10     if (!t.empty()) { // 空文字列でない?
11         std::cout << "size: " << t.size() << "\n";
12         std::cout << t[0] << " " << t[t.size()-1] << " ";
13         std::cout << t.front() << " " << t.back() << " ";
14         t.front() = 'X'; // 左辺値として
15         t.back() = 'Z';  // 左辺値として
16         std::cout << t << "\n";
17     }
18 }
```

```
% ./a.out
X abcdefg
hello
size: 5
h o h o XellZ
```

※下線部はキーボード入力

string 型のメンバ関数

- `s` はstring型の変数とする

メンバ関数

動作

`s.find(x)` `s` の先頭から `x` を探して添字を返す

`s.rfind(x)` `s` の末尾から `x` を探して添字を返す

`s.substr(x, y)` `s` の `x` 番目から `y` 個の文字（部分文字列）を返す

`s.replace(x, y, z)` `s` の `x` 番目から `y` 個の文字を `z` に置き換える

- `find()`, `rfind()` で見つからない場合、特殊な値 `std::string::npos` が返却される

```
1  #include <iostream>
2  int main() {
3      std::string s {"abcdefghijabc"};
4      int i, j, k;
5      i = s.find("b");    // 前から見たbの添字
6      j = s.find("cde");  // 前から見たcの添字
7      k = s.rfind("abc"); // 後ろから見たaの添字
8      std::cout << i << " " << j << " " << k << "\n";
9
10     if (s.find("X") == std::string::npos)
11         std::cout << "X is not found\n";
12
13     std::cout << s.substr(2, 3) << "\n"; // cde
14     std::cout << s.substr(9) << "\n";    // jabc
15
16     s.replace(10, 3, "klmn"); // abcdefghijklmn
17     std::cout << s << "\n";
18 }
```

```
% ./a.out
1 2 10
X is not found
cde
jabc
abcdefghijklmn
```

数値と文字列の相互変換

- `to_string(x)`: 数値である `x` を数字の文字列へ変換
- `stoi(x)`: 数字の文字列 `x` を `int` 値へ変換
- `stod(x)`: 数字の文字列 `x` を `double` 値へ変換

※ 文字列の後ろ側には数字以外の文字が含まれていてもよい

```
1  #include <iostream>
2
3  int main() {
4      std::string s {"i = "};
5      int i {10};
6      s += std::to_string(i);
7
8      double r {2.5};
9      s += ", r = " + std::to_string(r);
10     std::cout << s << "\n";
11
12     std::string ten{"10s"}, pi{"3.1415 rad"};
13     i = std::stoi(ten);
14     r = std::stod(pi);
15     std::cout << i << ", " << r << "\n";
16 }
```

```
% ./a.out
i = 10, r = 2.500000
10, 3.1415
```

vector型

vector 型

- 同一の型の値を複数同時に扱うためのデータ型
 - `char` も対象だが通常は `string` を使用する
- `<vector>` ヘッダファイルをインクルード
- 要素の型を宣言の際に指定する
- `n` 文字の各要素は配列の添字を使ってアクセス
 - 半開区間 `[0,n)`
- ※ `python` のリストと大体同じものです

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int>    x; // 初期化なしで宣言. int型.
6      std::vector<double> y; // 初期化なしで宣言. double型.
7
8      std::vector<int> b(3); // 要素3個, 初期値 0
9      std::cout << b[0] << " "
10                 << b[1] << " "
11                 << b[2] << "\n"; // 0 0 0
12
13      b[0] = 1; // 0番目の値の変更
14      b[2] = 3; // 2番目の値の変更
15
16      std::cout << b[0] << " "
17                 << b[1] << " "
18                 << b[2] << "\n"; // 1 0 3
19
20      x = b; // 3要素のコピー
21  }
```

vectorの宣言

- 丸括弧 `()` と中括弧 `{}` で、指定の意味が異なる
- 初期値を指定すると型名を省略できる

```
1  #include <iostream>
2  #include <vector>
3  int main() {
4
5      std::vector<double> a;      // 要素数0
6
7      std::vector<double> b(5);   // 要素数5, 初期値0.0
8
9      std::vector<double> c(5, 1.4); // 要素数5, 初期値1.4
10     std::vector c2(5, 1.4);      // cの省略形
11
12     std::vector<double> d {1.2, 2.5, 3.5}; // 中括弧の初期値指定
13     std::vector d2 {1.2, 2.5, 3.5};      // dの省略形
14 }
```

そもそも配列のような仕組みは、なぜ必要？

- 学生の成績を変数として保存することを考える
- 左右のコードは、同じ挙動。どちらが良いか？

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      int student_1{80}, student_2{93}, student_3{74}, student_4{53}, student_5{82};
6      const int n_student {5};
7
8      std::cout << "score:\n"
9              << " student 1: " << student_1 << "\n"
10             << " student 2: " << student_2 << "\n"
11             << " student 2: " << student_3 << "\n"
12             << " student 4: " << student_4 << "\n"
13             << " student 5: " << student_5 << "\n";
14
15      int total { student_1 + student_2 + student_3 + student_4 + student_5 };
16      double average { total / static_cast<double>(n_student) };
17      std::cout << "total: " << total << ", ave: " << average << "\n";
18
19  }
```

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> students {80, 93, 74, 53, 82};
6
7
8      std::cout << "score:\n"
9              << " student 1: " << students[0] << "\n"
10             << " student 2: " << students[1] << "\n"
11             << " student 2: " << students[2] << "\n"
12             << " student 4: " << students[3] << "\n"
13             << " student 5: " << students[4] << "\n";
14
15      int total { students[0] + students[1] + students[2] + students[3] + students[4] };
16      double average { total / static_cast<double>(students.size()) };
17      std::cout << "total: " << total << ", ave: " << average << "\n";
18
19  }
```

- 学生が5名程度であれば、使えなくはないが…
- 学生が増やすことも容易、`{}` の値を増やすだけ
- `for` 文（来週説明）と組み合わせることもできる

要素の追加と削除

- `v.push_back( x )` : vector `v` の末尾に要素 `x` の追加
- `v.pop_back()` : vector `v` の末尾要素の削除
- `v.size()` : vector `v` の要素数
- `v.clear()` : vector `v` の全要素の削除

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<double> x;
6      x.push_back(1.5);
7      x.push_back(2.8);
8      x.push_back(3.3);
9      std::cout << x.size() << ": " << x.front() << " " << x.back() << "\n";
10
11     x.pop_back();
12     std::cout << x.size() << ": " << x.front() << " " << x.back() << "\n";
13
14     x.clear();
15     std::cout << x.size() << "\n";
16 }
```

```
% ./a.out
3: 1.5 3.3
2: 1.5 2.8
0
```

vector 変数の比較

- 先頭の要素からそれぞれ比較. すべて同じならば `true`, 一つでも異なれば `false`.
- 各要素は比較可能でなければならない
 - ユーザ定義型の場合に注意

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector v1 { 1, 2, 3, 4, 5 };
6      std::vector v2 { 0, 2, 3, 4, 8 };
7      if (v1 == v2) std::cout << "equal\n";
8      if (v1 != v2) std::cout << "not equal\n";
9      if (v1 < v2) std::cout << "less than\n";
10     if (v1 <= v2) std::cout << "less than equal\n";
11     if (v1 > v2) std::cout << "greater than\n";
12     if (v1 >= v2) std::cout << "greater than equal\n";
13 }
```

```
% ./a.out
not equal
greater than
greater than equal
```

string 要素の vector

- 宣言時の省略に注意する
 - 文字列リテラルの初期値省略ではstring型とならない
 - `std::string` が関係する時には `<string>` を省略しない
- 配列内の文字へのアクセス
 - 2段階で取り出す: `vs[0][1]` → `(vs[0])[1]`

```
1  #include <iostream>
2  #include <vector>
3  using std::vector, std::string, std::cout;
4  int main() {
5      vector<string> vs {"abc", "def", "ghi"};
6      // vector vs {"abc", "def", "ghi"}; // これはやらないほうが良い.
7                                          // std::vector<char const*>と解釈されてしまうため.
8
9      s = vs[0];
10     cout << s[1] << " " << vs[0][1] << "\n"; // b b
11 }
```

vector の操作

メンバ関数	意味
<code>v.push_back(t)</code>	v の末尾に値tを追加する
<code>v.pop_back()</code>	v の末尾要素を削除する
<code>v.clear()</code>	v の全要素を削除する
<code>v[n]</code>	v の n 番目の要素
<code>v.size()</code>	v が持つ要素数を返す

メンバ関数	意味
<code>v.front()</code>	v の先頭要素
<code>v.back()</code>	v の末尾要素
<code>v.empty()</code>	v が空かどうか
<code>v1 = v2</code>	v1へのv2の上書き
<code>v1 == v2</code>	v1 と v2 が同一か
<code>v1 != v2</code>	v1 と v2 が同一でないか

pythonとの比較：vector

```
1 // cppのプログラム
2 #include <iostream>
3 #include <vector>
4 int main() {
5     std::vector<int> a;    // 要素数0
6     std::vector<int> b(5); // 要素数5, 初期値0
7     std::vector<double> c(5, 1.4); // 要素数5, 初期値1.4
8     std::vector<double> d {1.2, 2.5, 3.5}; // 各要素の値を指定する初期化
9     // std::cout << d << "\n"; // これはエラー!
10    std::cout << d[0] << " " << d[1] << " " << d[2] << "\n";
11
12    d.push_back(1.5);
13    d.push_back(2.8);
14    std::cout << d.size() << ": " << d.front() << " " << d.back() << "\n";
15    d.pop_back();
16    std::cout << d.size() << ": " << d.front() << " " << d.back() << "\n";
17    d.clear();
18    std::cout << d.size() << "\n";
19
20    std::vector<std::string> vs {"abc", "def", "ghi"};
21    std::string s = vs[0];
22    std::cout << s[1] << " " << vs[0][1] << "\n";
23 }
24 // ここまで
```

```
% ./a.out
1.2 2.5 3.5
5: 1.2 2.8
4: 1.2 1.5
0
b b
```

```
1 # pythonのプログラム
2
3
4
5 a = []
6 b = [0] * 5 # 要素数5, 初期値0.0
7 c = [1.4] * 5 # 要素数5, 初期値1.4
8 d = [1.2, 2.5, 3.5] # 各要素の値を指定する初期化
9 print( d ) # pythonでこれはOK
10 print( d[0], d[1], d[2] )
11
12 d.append(1.5)
13 d.append(2.8)
14 print( len(d), ":", d[0], " ", d[-1] )
15 d.pop()
16 print( len(d), ":", d[0], " ", d[-1] )
17 d.clear()
18 print(len(d))
19
20 vs = ["abc", "def", "ghi"]
21 s = vs[0]
22 print( s[1], vs[0][1] )
23
24 # ここまで
```

```
% python3 main.py
[1.2, 2.5, 3.5]
1.2 2.5 3.5
5 : 1.2 2.8
4 : 1.2 1.5
0
b b
```